CarbonWasteprint

ISTRUZIONI DI RIPRISTINO

Portale Carbon WastePrint®

Copie di sicurezza e ripristino di servizi, GitHub e Claude

DOCUMENTO	REVISIONE	DATA DI REVISIONE
Istruzioni di ripristino	Rev. 1.8	21/10/2026

Programma di

Fondazione operate

RISERVATO AGLI AMMINISTRATORI

Il documento

Queste istruzioni spiegano dove stanno il codice e i dati del portale, quali copie di sicurezza fare e come rimettere in funzione GitHub, Vercel, Supabase, Resend e il progetto Claude se qualcosa va perso.

Destinatari	Amministratori del portale e sviluppatori — documento riservato
Revisione corrente	Rev. 1.8 del 21/10/2026
Redatto da	Fondazione Operate
Versione sempre aggiornata	Area Documentazione del portale (solo Amministratori)

Registro delle revisioni

Rev.	Data	Descrizione	Autore
1.8	21/10/2026	Script 36 e funzione notizie (Verify JWT disattivato, indirizzo url_notizie, quarto lavoro pianificato).	Fondazione Operate
1.7	21/10/2026	Backup automatico su Google Drive alle 13 e alle 21 (installazione, contenuto) e ripristino da un backup automatico, provato sul banco locale.	Fondazione Operate
1.6	21/10/2026	Ricostruzione del database con gli script da 01 a 35 (simulatore, Amministratore principale, assistente, utenti online, Audit User, avvisi di sicurezza, controlli del collaudo).	Fondazione Operate
1.5	20/10/2026	Indirizzo Supabase nella Content-Security-Policy di vercel.json; script 01-34.	Fondazione Operate
1.4	20/10/2026	Amministratore principale e verifiche dopo il ripristino (Audit User, assistente).	Fondazione Operate
1.3	20/10/2026	Script 01-33.	Fondazione Operate
1.2	20/10/2026	Script 01-32.	Fondazione Operate
1.1	19/10/2026	Funzione assistente e chiave ANTHROPIC_API_KEY; Claude Console tra i servizi; script 01-31.	Fondazione Operate
1.0	19/10/2026	Prima emissione: dove stanno codice e dati, copie di sicurezza, ripristino di GitHub, Vercel, Supabase, Resend e del progetto Claude, elenco di chiavi e impostazioni.	Fondazione Operate

Sommario

01 A chi serve e quando usarlo 4

02 Dove stanno il lavoro e i dati 5

03 Copie di sicurezza: che cosa fare prima che serva 6

04 Backup automatico su Google Drive 8

05 GitHub e Vercel: tornare alla versione precedente 11

06 Ripristino di GitHub 12

07 Ripristino di Vercel 13

08 Supabase: progetto in pausa 14

09 Supabase: dati cancellati per errore 15

10 Supabase: ricostruzione da zero 16

11 Ripristino di Resend 19

12 Ripristino del progetto Claude 20

13 Elenco di chiavi e impostazioni 21

01

A chi serve e quando usarlo

Queste istruzioni servono a **rimettere in funzione il portale e a recuperare il lavoro** se qualcosa va perso: un servizio cancellato per errore, dati sbagliati nel database, un account chiuso, il progetto Claude eliminato.

Sono scritte per due lettori:

- **Luca o un amministratore della Fondazione**, che deve capire che cosa è successo e da dove ripartire;
- **lo sviluppatore** che fa il ripristino tecnico: i comandi sono nei riquadri di codice.

Da dove partire. Trovate il caso nella tabella e andate al capitolo indicato.

Che cosa è successo	Capitolo
Il portale non si apre dopo giorni di inattività	Supabase: progetto in pausa
Una modifica al portale ha rotto qualcosa	GitHub e Vercel: tornare alla versione precedente
Il repository GitHub è stato cancellato	Ripristino di GitHub
Il progetto Vercel è stato cancellato	Ripristino di Vercel
Dati cancellati o modificati per errore nel portale	Supabase: dati cancellati per errore
Il progetto Supabase è stato cancellato	Supabase: ricostruzione da zero
Le email non partono più	Ripristino di Resend
Il progetto Claude è stato cancellato	Ripristino del progetto Claude

DA SAPERE

Regola d'oro. Le chiavi e le password dei servizi non sono mai nei backup, nel codice o nei documenti. Le inserisce solo chi ne è titolare, direttamente nella dashboard del servizio. In questo documento compaiono solo i **nomi** delle chiavi.

02

Dove stanno il lavoro e i dati

Il portale non ha un server proprio: è distribuito su sei servizi. Sapere che cosa sta dove dice che cosa si perde e che cosa si recupera.

Servizio	Che cosa contiene	Se si perde
GitHub (lucamorettioperate/portale-cwp)	Tutto il codice, gli script del database (01-36), le funzioni del server, i manuali e la loro storia	Si recupera dal ramo `snapshot-…` o dal file `.bundle` dell'archivio di backup
Vercel (progetto portale-cwp)	Solo la pubblicazione del portale; nessun dato	Si ricrea in 10 minuti collegando di nuovo GitHub
Supabase (organizzazione Fondazione Operate, progetto portale-cwp)	Database (pratiche, utenti, audit, impostazioni), file caricati, funzioni del server, lavori pianificati, chiavi	La struttura si ricrea dagli script; **i dati e i file solo da una copia** (capitolo «Copie di sicurezza»)
Resend	Invio delle email; chiave API e verifica del dominio	Nuovo account o nuova chiave, poi aggiornare Supabase
Claude Console di Anthropic (organizzazione «Luca's Individual Org», spazio di lavoro dedicato)	Chiave API e crediti dell'assistente virtuale; nessun dato del portale	Nuova chiave legata a un solo spazio di lavoro, poi aggiornare `ANTHROPIC_API_KEY` in Supabase
Claude (progetto «CWP PORTAL CARBON WASTE PRINT»)	Documenti del progetto, memoria, artefatti, conversazioni	Si ricrea dall'archivio di backup (cartelle 5 e 6)

L'archivio di backup. Claude prepara su richiesta un file `Portale_CWP_backup_completo_<data>.zip` con:

1. `1-codice-github`: il repository con tutta la storia (`.bundle`) e il codice leggibile (`.zip`);
2. `2-manuali`: Manuale Utente, Amministratore, Test, Registro e queste istruzioni, in Word e PDF;
3. `3-presentazione`;
4. `4-documenti-ufficiali-CWP`: gli 8 PDF del Programma, che servono anche per ricostruire la pagina /programma;
5. `5-progetto-claude`: documenti del progetto, memoria, elenco delle conversazioni;
6. `6-artefatti`: gli artefatti pubblicati, in PDF;
7. `7-materiale-ricevuto` e `8-esempi-prodotti`.

L'archivio va tenuto **in due posti diversi**: il computer (meglio un disco esterno) e un cloud diverso da GitHub, per esempio Google Drive.

03

Copie di sicurezza: che cosa fare prima che serva

Un ripristino è possibile solo se esiste una copia. Questo è il calendario consigliato.

Quando	Che cosa	Chi
Ogni giorno alle 13 e alle 21	**Backup automatico completo su Google Drive** (cartella «Cwp»: codice, database, file, configurazione; una cartella per backup)	Google Apps Script di Luca, da solo (capitolo «Backup automatico su Google Drive»)
Ogni settimana	Togliere a mano le cartelle di backup vecchie (tenerne almeno 14, cioè una settimana)	Luca
A ogni rilascio importante	Archivio di backup completo (chiederlo a Claude) e ramo `snapshot-<data>` su GitHub	Claude, su richiesta
Ogni settimana, appena ci sono dati reali	Copia del database e dei file (riquadri qui sotto)	Sviluppatore
Ogni giorno, dopo il passaggio al piano Pro	Copia automatica del database, tenuta 7 giorni	Supabase, da solo
Ogni tre mesi	Prova di ripristino su un progetto Supabase di prova	Sviluppatore
Sempre	Chiavi e password nel gestore di password della Fondazione	Titolari degli account

Database

Oggi il progetto Supabase è sul **piano gratuito, che non fa copie automatiche**. Supabase stesso consiglia, per questo piano, di esportare regolarmente i dati con il proprio strumento a riga di comando (Supabase CLI). La stringa di connessione si trova in Supabase, pulsante *Connect*; contiene la password del database, che va digitata solo da chi la conosce e non va mai salvata nei file.

```
supabase db dump --db-url "<stringa di connessione>" -f roles.sql --role-only
supabase db dump --db-url "<stringa di connessione>" -f schema.sql
supabase db dump --db-url "<stringa di connessione>" -f data.sql --use-copy --data-only
```

I tre file (ruoli, struttura, dati) vanno conservati insieme all'archivio di backup, in una cartella con la data.

Con il piano Pro Supabase fa una copia al giorno e la tiene 7 giorni; con l'opzione *Point-in-Time Recovery* si torna a un minuto preciso degli ultimi 7 giorni. La copia manuale settimanale resta utile: è l'unica che si può scaricare e tenere fuori da Supabase.

File caricati (archivio Storage)

DA SAPERE

ATTENZIONE

Attenzione. Le copie del database di Supabase, anche quelle automatiche del piano Pro, **non contengono i file**: contengono solo l'elenco dei file. I documenti caricati vanno copiati a parte.

Gli archivi del portale sono tre:

Archivio	Contenuto
`documenti`	I documenti delle pratiche caricati da Comuni e Fondazione
`modelli`	I modelli di documento e gli 8 documenti ufficiali del Programma
`allegati-consultazione`	Gli allegati alle osservazioni del feedback pubblico

Con la Supabase CLI, collegata al progetto, si copia ogni archivio in una cartella locale:

```
supabase storage cp -r ss:///documenti ./file/documenti --experimental
supabase storage cp -r ss:///modelli ./file/modelli --experimental
supabase storage cp -r ss:///allegati-consultazione ./file/allegati-consultazione --experimental
```

Finché i file sono pochi si possono anche scaricare dalla dashboard: *Storage*, archivio, selezione dei file, *Download*.

Chiavi e impostazioni

Le chiavi non si copiano: si **rigenerano** nel servizio. Tenete invece aggiornato l'elenco del capitolo «Elenco di chiavi e impostazioni», così chi ripristina sa che cosa ricreare.

04
Backup automatico su Google Drive

Dal 21/10/2026 (richiesta di Luca) un programma in **Google Apps Script**, nell'account Google di Luca, fa **ogni giorno due backup completi**, tra le 13 e le 14 e tra le 21 e le 22 (ora italiana). Ogni backup è una **nuova cartella** dentro la cartella **Cwp** del Drive di Luca, con il nome «Backup_CWP_aaaa-mm-gg_hhmm». Le cartelle vecchie non si cancellano da sole: le toglie Luca a mano.

Cartella	Contenuto
`1_codice`	ZIP del codice da GitHub (ramo main): portale, script del database 01-36, funzioni del server, manuali, strumenti di ripristino; `commit.json` con il commit di riferimento
`2_database`	Un file JSON per ogni tabella del database, `auth_utenti.json` (utenti con id ed email, **senza password**: non si possono esportare) e `conteggi.json`
`3_file`	Tutti i file degli archivi `documenti`, `modelli` e `allegati-consultazione`, con le stesse sottocartelle, ed `elenco_file.json`
`4_servizi`	Configurazione di Supabase, Vercel e Resend: nomi delle funzioni e dei segreti, impostazioni di Auth, variabili di Vercel, domini Resend. **Mai i valori segreti**
`LEGGIMI.txt`, PDF delle Istruzioni di ripristino, `esito.json`	Guida rapida, queste istruzioni aggiornate ed esito del backup

Se un backup non riesce o non finisce (Google concede al massimo 6 minuti per esecuzione), Luca riceve un'email «Backup CWP NON completo» con gli errori.

Il backup contiene dati personali (email degli utenti, documenti delle pratiche): la cartella Cwp non va condivisa.

Installazione (una volta, la fa Luca: circa 10 minuti)

Il codice del programma è nel repository, `strumenti/backup/BackupCWP.gs`: si apre in testo semplice all'indirizzo https://raw.githubusercontent.com/lucamorettioperate/portale-cwp/main/strumenti/backup/BackupCWP.gs. Nella cartella Cwp c'è anche il documento «Backup CWP — installazione» con questi passi.

1. Aprite **script.google.com** con l'account Google che possiede la cartella Cwp → **Nuovo progetto**. Rinominatelo «Backup CWP».
2. Aprite l'indirizzo del codice qui sopra, selezionate tutto (Cmd+A / Ctrl+A) e copiate. Nell'editor cancellate il contenuto del file `Codice.gs`, incollate e premete **Salva** (icona del dischetto).

3. **Impostazioni progetto** (ingranaggio a sinistra): - **Fuso orario**: «(GMT+01:00) Roma» (oppure «Europe/Rome»); - **Proprietà dello script** → **Aggiungi proprietà dello script**, nome `SUPABASE_SERVICE_ROLE_KEY`. Il valore è in Supabase → *Project Settings* → *API Keys* → chiave **service_role**: copiatela e incollatela voi, non va mai scritta altrove; - facoltative: `RESEND_API_KEY` (per salvare l'elenco dei domini Resend) e `VERCEL_TOKEN` (per salvare le impostazioni del progetto Vercel).
4. Tornate all'editor, scegliete la funzione **installa** nel menu in alto e premete **Esegui**. Google chiede l'autorizzazione: scegliete l'account, *Avanzate* → *Vai a Backup CWP* → **Consenti** (servono Drive, connessioni esterne ed email).
5. Scegliete la funzione **provaOra** e premete **Esegui**: dopo uno o due minuti nella cartella Cwp compare il primo «Backup_CWP_...». Aprite `esito.json`: deve dire `"completo": true`.
6. In **Attivatori** (sveglia a sinistra) devono comparire due attivatori giornalieri di `eseguiBackup`, alle 13 e alle 21.

Se la chiave service_role di Supabase cambia, aggiornate la proprietà dello script. Per fermare i backup cancellate i due attivatori.

Ripristino da un backup automatico

Si usa quando dati o progetto Supabase sono persi. Serve un computer con Node 18+ e, per i dati grandi, `psql` (lo sviluppatore).

1. **Scegliete il backup**: l'ultima cartella con `esito.json` → `"completo": true`. Scaricatela dal Drive (tasto destro → *Scarica*) e scompattate lo ZIP di `1_codice`.
2. **Nuovo progetto Supabase** ed esecuzione degli script 01-36, come nel capitolo «Supabase: ricostruzione da zero», passi 1 e 2.
3. **Dati**: dalla cartella del codice scompattato eseguite `node strumenti/backup/genera_sql_ripristino.mjs <cartella 2_database> ripristino_dati.sql`, poi `psql "<stringa di connessione del nuovo progetto>" -f ripristino_dati.sql`. Se il file è piccolo si può incollare nel SQL Editor. - Il file rimette utenti, pratiche, documenti, calcoli, audit e impostazioni **identici**, con gli stessi codici, senza far partire notifiche o voci di audit nuove. - Si ferma da solo se il database contiene già delle pratiche, così non si sovrascrive per errore il portale in uso. - In fondo mostra, tabella per tabella, le righe nel backup e quelle ripristinate. Un «RIGA NON RIPRISTINATA» indica un dato vecchio che non rispetta un controllo attuale: va valutato a parte.
4. **File**: `SUPABASE_URL=… SUPABASE_SERVICE_ROLE_KEY=… node strumenti/backup/carica_file_ripristino.mjs <cartella 3_file>`. La chiave è quella del nuovo progetto e la inserisce chi esegue il ripristino.
5. **Funzioni del server e segreti**: dalla cartella `supabase/functions` del codice, con i segreti elencati in `4_servizi/configurazione_servizi.json` (capitolo «Chiavi delle funzioni»).
6. **Autenticazione**, **Vercel** e **Resend**: come nei capitoli di questo documento, con i valori di `4_servizi`.

7. **Utenti**: tornano con lo stesso id, la stessa email e lo stesso ruolo, ma senza password. Ognuno, al primo accesso, usa «Password dimenticata?». In alternativa l'Amministratore usa «Reimposta password».
8. **Verifica**: accesso dell'Amministratore principale, una pratica con i documenti, l'Audit, il calcolo dei crediti, una notifica.

La procedura dei dati è stata provata il 21/10/2026 sul banco locale. Su un database ricostruito con gli script 01-35 sono state ripristinate 35 tabelle e 3 utenti, con dati identici al JSON. La prova si è fermata da sola alla seconda esecuzione; un dato vecchio non valido è stato segnalato e i contatori sono stati riallineati. Da ripetere ogni tre mesi su un progetto Supabase di prova (tabella «Copie di sicurezza»).

05

GitHub e Vercel: tornare alla versione precedente

Se un rilascio ha introdotto un errore, il modo più rapido è **ripubblicare la versione precedente**:

1. Su vercel.com aprite il progetto **portale-cwp**, scheda *Deployments*.
2. Trovate l'ultimo rilascio che funzionava (data e messaggio della modifica).
3. Dal menu del rilascio scegliete *Promote to Production* (o *Instant Rollback*). In un minuto il portale torna com'era.

Poi lo sviluppatore corregge il codice con calma: la storia completa è su GitHub e ogni versione fotografata ha un ramo `snapshot-<data>` (il primo è `snapshot-2026-10-19`, la versione della presentazione).

> **DA SAPERE**
> Se l'errore riguarda il **database** (uno script eseguito), tornare indietro su Vercel non basta: serve uno script di correzione o il ripristino dei dati (capitolo Supabase).

06

Ripristino di GitHub

Repository cancellato da meno di 90 giorni

GitHub conserva i repository cancellati per 90 giorni (possono servire fino a un'ora prima che compaiano nell'elenco):

1. Su github.com, con l'account **lucamorettioperate**: foto del profilo › *Settings* › *Repositories* › *Deleted repositories*.
2. Accanto a **portale-cwp** premete *Restore* e confermate con *I understand, restore this repository*.
3. Su Vercel verificate che il progetto sia ancora collegato (scheda *Settings* › *Git*); se no, ricollegatelo.

Se il repository fa parte di un'organizzazione GitHub, il percorso è: *Organizations* › organizzazione › *Settings* › *Deleted repositories*.

Repository perso definitivamente

Si ricrea dal file `portale-cwp-storia-completa.bundle` dell'archivio di backup, che contiene tutte le versioni:

1. Su github.com create un repository **vuoto** con lo stesso nome (`portale-cwp`), senza README.
2. Lo sviluppatore, nella cartella dove ha estratto il backup, esegue:

```
git clone portale-cwp-storia-completa.bundle portale-cwp
cd portale-cwp
git remote set-url origin https://github.com/lucamorettioperate/portale-cwp.git
git push origin --all
```

Infine, su Vercel ricollegate il progetto al repository (capitolo successivo).

Se su Google Drive il bundle è diviso in due parti (`parte0` e `parte1`), si riuniscono prima di tutto:

```
cat portale-cwp-storia-completa.bundle.parte0 \
    portale-cwp-storia-completa.bundle.parte1 \
    > portale-cwp-storia-completa.bundle
```

07

Ripristino di Vercel

Vercel non contiene dati: se il progetto è cancellato, si ricrea.

1. Su vercel.com: *Add New* › *Project* › importate il repository **portale-cwp** da GitHub.
2. Impostazioni di build (di solito le riconosce da solo): framework **Vite**, comando `npm run build`, cartella `dist`. Il file `vercel.json` del repository fa già funzionare i collegamenti diretti alle pagine (per esempio /programma).
3. *Environment Variables*: inserite `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY`. I valori si leggono in Supabase, *Project Settings* › *API*; li inserisce Luca.
4. *Deploy*. Al termine controllate che il portale si apra e che l'accesso funzioni.
5. Se l'indirizzo del portale cambia (nuovo nome o dominio, per esempio portale.operate.it), aggiornatelo anche in Supabase: *Authentication* › *URL Configuration* (Site URL e Redirect URL) e la chiave `PORTALE_URL` delle funzioni.

08

Supabase: progetto in pausa

Sul piano gratuito Supabase mette in pausa il progetto dopo una settimana senza utilizzo. I dati non si perdono:

1. Su supabase.com aprite l'organizzazione **Fondazione Operate** e il progetto **portale-cwp**.
2. Premete *Resume project* (o *Restore project*) e attendete qualche minuto.

Secondo la documentazione di Supabase un progetto gratuito in pausa si può riattivare entro un anno. Con il piano Pro il progetto non va mai in pausa.

09

Supabase: dati cancellati per errore

Il portale non cancella quasi nulla (documenti versionati, audit non modificabile), quindi il caso più comune è un **dato modificato per errore**: si corregge dal portale stesso o con uno script di correzione dello sviluppatore, che resta tracciato nell'audit.

Se servono i dati di un momento precedente:

- **Piano Pro**: *Database* › *Backups*, scegliete la copia del giorno più vicino **prima** dell'errore e confermate. Durante il ripristino il portale non è raggiungibile e **tutto** il database torna a quel giorno: le modifiche fatte dopo vanno rifatte a mano. Con l'opzione *Point-in-Time Recovery* si sceglie il minuto esatto.
- **Piano gratuito**: non ci sono copie automatiche; si usa l'ultima copia manuale (capitolo «Copie di sicurezza»), meglio caricandola su un progetto di prova e riportando solo i dati che servono.

> **DA SAPERE**
> Prima di ogni ripristino fate una copia dello stato attuale: se il ripristino non va come previsto, si può tornare indietro.

10
Supabase: ricostruzione da zero

Se il progetto Supabase è stato cancellato, lo sviluppatore lo ricostruisce in quest'ordine. Le chiavi le inserisce Luca.

1. Nuovo progetto

1. Su supabase.com, nell'organizzazione **Fondazione Operate**: *New project*, nome **portale-cwp**, regione europea, password del database generata e salvata nel gestore di password.
2. *Database* › *Extensions*: attivate **pg_cron** e **pg_net** (servono ad avvisi, scadenze ed email automatiche).

2. Struttura del database (script)

Nel *SQL Editor* eseguite gli script della cartella `supabase/` del repository, **uno alla volta e in ordine di numero**, da `01_schema.sql` a `36_notizie.sql`:

- **non** eseguite `schema_completo.sql`: è una vecchia fotografia, superata dagli script numerati;
- `03_dati_esempio.sql` crea solo dati di prova (Comune di Borgoval): eseguitelo solo per un ambiente di prova;
- se compare l'avviso *Potential issue detected*, confermate con *Run this query*: gli script sono pensati per essere rieseguibili;
- **prima** di `25_documenti_programma.sql` caricate gli 8 PDF della cartella `4-documenti-ufficiali-CWP` nella radice dell'archivio **modelli** (*Storage* › modelli › *Upload*), con gli stessi nomi di file;
- gli script dal 28 al 35 aggiungono simulatore, Amministratore principale (29), eliminazione degli utenti mai attivi, assistente, utenti online, Audit User, avvisi di sicurezza (34) e i controlli dei dati chiesti dal collaudo (35) e l'osservatorio News (36); il 35 termina con un conteggio di sola lettura che su un database nuovo vale 0 in tutte le righe;
- **dopo** `12_email_notifiche.sql` aggiornate l'indirizzo della funzione email con quello del nuovo progetto:

```
update impostazione
   set valore = 'https://<nuovo-progetto>.supabase.co/functions/v1/invia-email-notifica'
 where chiave = 'url_email_notifiche';
```

- **dopo** `36_notizie.sql`, allo stesso modo, l'indirizzo della funzione delle News:

```
update impostazione
   set valore = 'https://<nuovo-progetto>.supabase.co/functions/v1/notizie'
 where chiave = 'url_notizie';
```

3. Dati e file (se esiste una copia)

Se avete la copia manuale del database (roles.sql, schema.sql, data.sql), lo sviluppatore la carica **al posto** degli script del passo 2, con la stringa di connessione del nuovo progetto:

```
psql --single-transaction --variable ON_ERROR_STOP=1 \
  --file roles.sql --file schema.sql \
  --command 'SET session_replication_role = replica' \
  --file data.sql --dbname "<stringa di connessione>"
```

Poi ricaricate i file nei tre archivi (`documenti`, `modelli`, `allegati-consultazione`) **con gli stessi percorsi**, per esempio:

```
supabase storage cp -r ./file/documenti ss:///documenti --experimental
```

Senza una copia dei dati il portale riparte vuoto: struttura, impostazioni e pagina /programma tornano dagli script; pratiche, utenti e documenti vanno ricaricati.

4. Funzioni del server (Edge Functions)

Create le sette funzioni con lo **stesso nome**, incollate il codice da `supabase/functions/<nome>/index.ts` e premete *Deploy*. Dopo la pubblicazione controllate l'impostazione **Verify JWT**:

Funzione	Verify JWT
`crea-utente`	Attivo
`reimposta-password`	Attivo
`assistente`	Attivo
`consultazione-pubblica`	**Disattivato** (pagina pubblica senza accesso)
`recupera-password`	**Disattivato** (chi l'ha dimenticata non è entrato)
`invia-email-notifica`	**Disattivato** (la chiama il database)
`notizie`	**Disattivato** (la chiama il lavoro pianificato ogni 2 ore; si protegge da sola: al massimo una ricerca ogni 100 minuti senza accesso di un Amministratore)

5. Chiavi delle funzioni (le inserisce Luca)

Edge Functions › *Secrets*:

- `RESEND_API_KEY`: chiave API di Resend;
- `EMAIL_MITTENTE`: il mittente sul dominio verificato (dopo la verifica del dominio);
- `PORTALE_URL`: l'indirizzo del portale, se diverso da portale-cwp.vercel.app;
- `ANTHROPIC_API_KEY`: chiave della Claude Console per l'assistente virtuale e per il filtro delle News. Deve essere legata a **un solo spazio di lavoro** (workspace): una chiave «per tutti gli spazi di lavoro» viene rifiutata. L'organizzazione della Console deve avere credito (crediti mensili del piano Max collegato oppure crediti acquistati).

`SUPABASE_URL`, `SUPABASE_ANON_KEY` e `SUPABASE_SERVICE_ROLE_KEY` le fornisce Supabase da solo.

6. Autenticazione

1. *Authentication* › *URL Configuration*: Site URL `https://portale-cwp.vercel.app`, Redirect URL `https://portale-cwp.vercel.app/**`.
2. *Authentication* › *Emails* › SMTP personalizzato: server `smtp.resend.com`, porta 465, utente `resend`, password = chiave API di Resend (la inserisce Luca); mittente come in `EMAIL_MITTENTE`.

7. Primo amministratore

Se non avete ricaricato una copia dei dati, gli utenti non esistono più:

1. *Authentication* › *Users* › *Add user*: email e password di Luca.
2. Nel *SQL Editor*:

```
update profilo set ruolo = 'admin'
 where id = (select id from auth.users where email = '<email di Luca>');
```

Se lo script 29 è stato eseguito quando l'account di Luca non c'era ancora, rieseguitelo ora: segna luca.moretti@operate.it come **Amministratore principale** (protetto, con la sezione Audit User e il pannello «n online»).

Gli altri utenti li crea poi l'Amministratore dal portale (*Gestione utenti*).

8. Collegare il portale e verificare

1. Su Vercel aggiornate `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` con i valori del nuovo progetto. Nel file `vercel.json` del repository sostituite l'indirizzo del vecchio progetto (`ajxrnofgunynkynozmrn.supabase.co`, 4 volte) con quello nuovo, nella regola `Content-Security-Policy`: altrimenti il browser blocca i collegamenti al database e il portale non funziona. Poi ripubblicate (*Deployments* › *Redeploy*).
2. Verifiche: accesso come Amministratore principale, con il menu completo (fino ad Audit User) e la scheda Accessi di Audit User che registra l'accesso appena fatto; la pagina /programma mostra gli 8 documenti, la PEC e il soggetto promotore; un utente di prova riceve l'email; l'assistente risponde; nella pagina News «Aggiorna ora» completa una ricerca; i lavori pianificati sono quattro:

```
select jobname, schedule from cron.job order by jobname;
-- attesi: cwp_avvisi_crediti, cwp_notifiche_scadenze, cwp_notizie, cwp_riprova_email
```

Infine annotate il ripristino nel **Registro sviluppo e deploy** (cronologia), con data, causa e copia usata.

11

Ripristino di Resend

1. Su resend.com create una nuova chiave API (*API Keys*); se l'account è perso, create un nuovo account.
2. Inserite la chiave (lo fa Luca) in Supabase: *Edge Functions* › *Secrets* › `RESEND_API_KEY` e come password SMTP in *Authentication* › *Emails*.
3. Se il dominio **operate.it** era verificato: *Domains* › *Add domain*, poi inserite nel DNS di operate.it i record indicati da Resend (SPF, DKIM, eventuale DMARC) e attendete la verifica. Finché il dominio non è verificato le email arrivano solo al titolare dell'account Resend.
4. Prova: «Password dimenticata?» dalla pagina di accesso con un utente di prova.

12
Ripristino del progetto Claude

Il progetto Claude contiene il contesto del lavoro, non il portale: anche se va perso, il portale continua a funzionare. Per ripartire con Claude come prima:

1. **Nuovo progetto.** Su claude.ai create un progetto con lo stesso nome, «CWP PORTAL CARBON WASTE PRINT».
2. **Documenti del progetto.** Caricate nei file del progetto i documenti della cartella `5-progetto-claude` dell'archivio (`registro-dev-deploy.md`, `analisi-requisiti-vs-portale.md`) e, se utili, i manuali in PDF della cartella `2-manuali`.
3. **Istruzioni del progetto.** Riportate le regole di lavoro: risposte in italiano; a ogni rilascio si aggiornano Registro, Manuale Utente, Manuale Amministratore e Manuale Test; codice commentato in italiano; chiavi e password le inserisce solo Luca.
4. **Memoria.** In una chat del nuovo progetto allegate `memoria_del_progetto.md` e chiedete a Claude di importarla nella memoria del progetto.
5. **Codice.** Nella prima sessione di sviluppo chiedete a Claude di collegare il repository GitHub `lucamorettioperate/portale-cwp`: da lì Claude ritrova codice, script e Registro.
6. **Artefatti.** I PDF della cartella `6-artefatti` sono la copia di lettura. Per riaverli modificabili, allegate il PDF in una chat e chiedete a Claude di ricrearlo come documento. L'elenco con i link originali è in `ELENCO_ARTEFATTI.txt`.
7. **Conversazioni.** Non si possono reimportare. Per conservarle: su claude.ai, dal computer, *Impostazioni* › *Privacy* › *Esporta dati*; arriva per email un link per scaricarle, valido 24 ore. La storia tecnica completa del lavoro è comunque nel Registro sviluppo e deploy.

13
Elenco di chiavi e impostazioni

Solo i nomi e dove si trovano. I valori stanno nel gestore di password della Fondazione.

Dove	Nome	A che cosa serve
Vercel › Environment Variables	`VITE_SUPABASE_URL`	Indirizzo del progetto Supabase
Vercel › Environment Variables	`VITE_SUPABASE_ANON_KEY`	Chiave pubblica di Supabase (protetta dalle regole RLS)
Supabase › Edge Functions › Secrets	`RESEND_API_KEY`	Invio delle email
Supabase › Edge Functions › Secrets	`EMAIL_MITTENTE`	Mittente sul dominio verificato
Supabase › Edge Functions › Secrets	`ANTHROPIC_API_KEY`	Assistente virtuale (chiave della Claude Console legata a un solo spazio di lavoro)
Supabase › Edge Functions › Secrets	`ASSISTENTE_LIMITE_GIORNO`, `ASSISTENTE_MODELLO`	Facoltativi: domande al giorno per utente (40) e modello (Claude Haiku 5.5)
Supabase › Edge Functions › Secrets	`PORTALE_URL`	Indirizzo del portale nei link delle email (facoltativo)
Supabase › Authentication › Emails	Password SMTP	La chiave API di Resend
Supabase › Authentication › URL Configuration	Site URL e Redirect URL	Ritorno al portale dai link delle email
Supabase › database, tabella `impostazione`	`url_email_notifiche`	Indirizzo della funzione `invia-email-notifica`
Supabase › database, tabella `impostazione`	`mfa_obbligatoria`	Verifica in due passaggi per lo staff (oggi spenta)
Supabase › Project Settings	Password del database	Copie manuali e ripristini
GitHub, Vercel, Supabase, Resend	Password e verifica in due passaggi degli account	Accesso ai servizi

Account e titolari. GitHub: lucamorettioperate. Supabase: organizzazione Fondazione Operate. Vercel e Resend: account di Luca. Con il passaggio ai piani a pagamento conviene intestare tutti gli account alla Fondazione, con almeno due amministratori per servizio.